

Smart Gear — Milestone 1 Report

Problem, Team, and Experiments

Mechanics at independent shops spend too much time on diagnostics. They read OBD-II fault codes, manually look up TSBs, check repair history, and interpret sensor data across separate tools. Smart Gear is a platform that connects to a vehicle's OBD-II port and uses an LLM to pull all of this together into a single diagnostic report with actionable next steps.

Team: Solo project. I handle architecture, backend (Node.js/Fastify), React Native app, ML pipeline, LLM integration, and infrastructure (AWS, Docker, Terraform).

AI role: Claude LLM generates diagnostic reports (product feature) and assists development (Claude Code for scaffolding and architecture). Experiments measure AI's latency, cost, and reliability tradeoffs.

Project Plan and Progress

Week	Tasks	Status
1	Implement tool functions + agentic loop; async vs. sync LLM load test	In Progress
2	MCP server; circuit breaker experiment; fixed vs. agentic comparison; report + video	Planned

Done so far: obd-core edge module working (BLE + PID parsing), monolithic Fastify backend running (PostgreSQL), literature review complete (8 papers)

AI cost/benefit: Claude Code saves ~40–60% on scaffolding time. Risk: over-reliance on generated code, mitigated by manual review.

Objectives

Short-term: End-to-end prototype (OBD scan → AI report), three experiments with results, real-vehicle test at Howe Sound Automotive (Bowen Island).

Long-term: Commercial product for independent shops. Cost control via token budgets and response caching. Reliability via circuit breakers and BullMQ retries with dead-letter queues.

Related Work

Academic: XGBoost/Random Forest achieve 85–92% on DTC classification but struggle with multi-fault cases. AUTODIAG (2023) is closest to Smart Gear but has no LLM explanations. Commercial: FIXD shows raw codes only; Autel MaxiSys is \$2,000+ and dealer-focused; MUCAR uses DeepSeek but without vehicle-specific context. Smart Gear's agentic approach (LLM dynamically queries repair history, TSBs, ML predictions) is not offered by any current tool.

Methodology

Current system: Monolithic Fastify server. PostgreSQL for vehicle/repair data, Redis + BullMQ for async jobs. React Native app connects to OBD-II adapter via BLE, sends parsed data to the backend.

Fixed pipeline (baseline): Every scan runs ML model + LLM in parallel, merges results. Never checks TSBs or repair history.

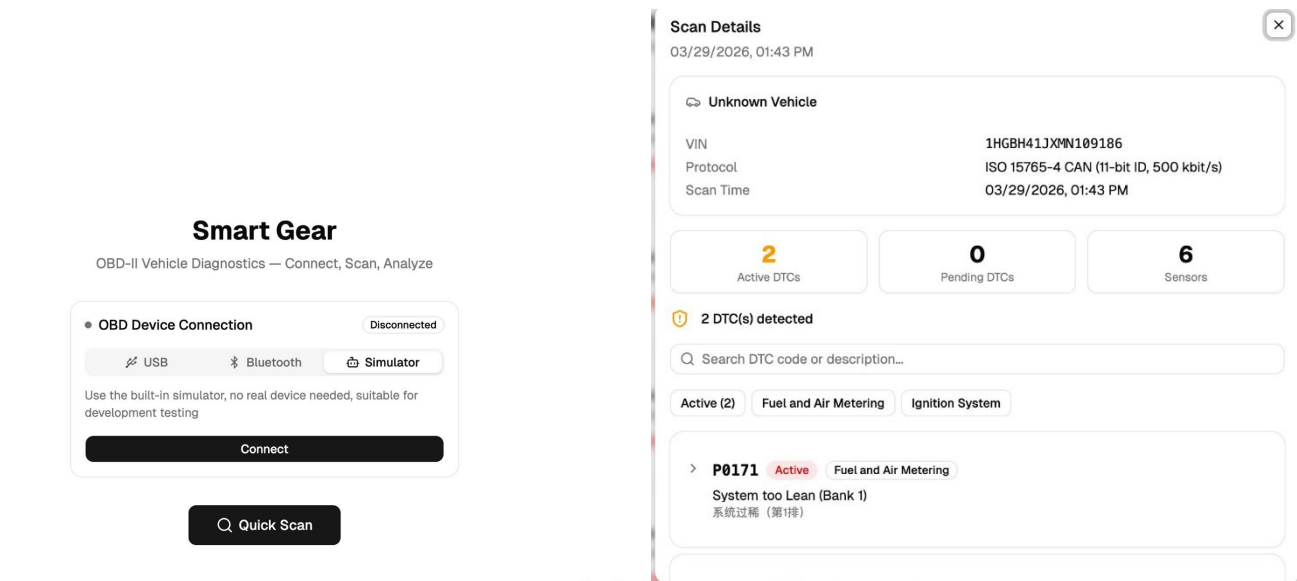
Agentic loop (experiment): LLM receives scan data and decides which tools to call: `run_ml_prediction()`, `lookup_dtc()`, `get_tsb()`, `query_repair_history()`, `detect_anomalies()`. Exposed as an MCP server so any compatible client can use the tools.

Exp 1 — Fixed vs. Agentic: 50+ scenarios, measure accuracy, latency, token cost.

Exp 2 — Async vs. Sync: Locust load test (10–500 users), measure throughput and P95 latency.

Preliminary Results

No formal experiment results yet. The agentic loop and MCP server are designed but not implemented — that is Week 1 of the sprint. What I can report from development so far: OBD-II PID parsing runs in sub-10ms on-device. Worst case workload: 100+ simultaneous scans with 4–5 tool calls each. Base case: single vehicle, single DTC, target under 3 seconds.



Impact

Reduces diagnostic research time for mechanics. For the course: practical data on fixed vs. agentic AI orchestration tradeoffs. The MCP server shows how standardized protocols decouple AI tools from specific apps. Classmates with an ELM327 adapter (\$10–20) can test by scanning their own vehicles.